

普及组初赛模拟 21

一、单项选择题 (共 15 题, 每题 2 分, 共计 30 分, 每题有且仅有一个正确选项)

1. 十六进制数 $(-73)_{16}$ 的 8 位二进制补码是()。
A. 10001110 B. 10001101 C. 01110010 D. 01110011
2. 以下关于 $CSP-J/S$ 的描述错误的是()。
A. $CSP-J/S$ 非专业认证在校生与在职人员皆可参加
B. $CSP-J/S$ 是中国计算机学会举办的程序设计竞赛
C. $CSP-J/S$ 属于 NOI 系列赛事
D. $CSP-J/S$ 认证者可在认证完成离开考场后重新返回考场
3. 以下语言中, 同时符合面向对象和解释执行的是什么语言()。
A. *Python* B. *C* C. *C++* D. *Pascal*
4. 小明家买了一台 4K 超清电视, 其屏幕分辨率是 4096×2160 , 每一个像素都是 32 位真彩色, 一个视频文件有 2 分钟, 每秒钟播放 24 帧。在没有压缩的情况下, 这个视频占用空间最接近以下哪个值()。
A. 80GB B. 25GB C. 100GB D. 200GB
5.
 1. 假设一个待排序数组已基本有序, 以下排序算法效率最高的是()。
A. 插入排序 B. 快速排序 C. 堆排序 D. 选择排序
6. 一个包含 m 个分支结点 (非叶子结点, 也就是度不为 0 的节点) 的非空满 n 叉树, $n \geq 1$, 它的叶结点数目为()。
A. $mn + 1$ B. $mn - 1$ C. $nm + m - 1$ D. $nm - m + 1$
7. 前缀表达式 $- * a + bcd$ 的后缀表达式是()。
A. $abcd * + -$ B. $abc + * d -$ C. $abc * + d -$ D. $- + * abcd$
8. 关于 CPU 的说法如下哪个是正确的?()
A. CPU 全称为中央计算器
B. *Intel* 公司发明了世界上最早的 CPU
C. CPU 能直接运行机器语言

D. 同样主频下, 64 位的 *CPU* 比 32 位的 *CPU* 运行速度快一倍

9. 关于计算机网络, 下面的说法哪些是正确的?()

- A. 网络协议之所以分层主要是新技术需要兼容历史上老的实现方案
- B. 新一代互联网使用的 *IPv5* 标准是 *IPv4* 标准的升级与补充
- C. *TCP/IP* 是互联网的基础协议簇, 包含有传输层和网络层的通信协议
- D. 互联网上每一台主机都需要注册一个固定的域名来对应其 *IP* 地址

10. 设哈希表的地址空间为 0 到 10, 散列函数为 $hash(n) = n \bmod 11$, 用线性探查法解决碰撞。现从空的哈希表开始, 依次插入关键码值 84, 25, 38, 57, 71, 则最后一个关键码 71 的地址为()。

- A. 7
- B. 6
- C. 5
- D. 4

11. 十进制数 0.25 等于十六进制数的多少?()

- A. 0.10
- B. 0.80
- C. 0.64
- D. 0.40

12. 以下逻辑表达式恒为真的是?()

- A. $A \vee (\neg A \wedge B) \vee (A \wedge B)$
- B. $B \vee (\neg A \wedge B) \vee (A \wedge \neg B)$
- C. $A \vee B \vee (\neg A \wedge B) \vee (A \wedge \neg B)$
- D. $A \vee \neg B \vee (\neg A \wedge B) \vee (A \wedge \neg B)$

13. 已知 *rand()* 可以生成一个 0 到 32767 的随机整数, 如果希望得到一个范围在 $[m, n]$ 的随机整数, *m* 和 *n* 均是不超过 1024 的正整数且 $m < n$, 那么可行的表达式是什么?()

- A. $rand() \% (n - m) + m$
- B. $rand() \% (n - m + 1) + m$
- C. $rand() \% (n - m) + m + 1$
- D. $rand() \% (n - m + 1) + m + 1$

14. *CSP-J* 初赛结束后, 某学校 1 位教练带 4 名女生和 5 名男生排成一行拍照留念, 要求男生必须排在一起, 老师在中间, 女生也必须排在一起, 那么不同的排列方式共有()种。

- A. 2880
- B. 5760
- C. 8640
- D. 11520

15. 一张有 10 个结点的非连通无向图最多有()条边。

- A. 40
- B. 81
- C. 72
- D. 36

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填 $\sqrt{\quad}$, 错误填 *X*; 除特殊说明外, 判断题 1.5 分, 选择题 3 分, 共计 40 分)

1.

```
1 #include <iostream>
2 using namespace std;
```

```

3 int main()
4 {
5     int a, b, c;
6     cin >> a >> b >> c;
7     a = a * b;
8     b = a / b;
9     a = a / b;
10    c = a / b;
11    cout << a << " " << b << " " << c << endl;
12    return 0;
13 }

```

xxx 判断题

1. 1) 若输入 2 4 1, 则输出 4 2 2。 ()
2. 2) 若输入 1234567890 8 5, 则输出 a 和 b 分别是 8 和 1234567890。 ()
3. 3) 将第 1 行头文件 `#include <iostream>` 替换成 `#include <bits/stdc++.h>`, 程序运行结果不会改变。 ()
4. 4) 将第 1 行头文件 `#include <iostream>` 替换成 `#include <cstdio>`, 第 11 行替换为 `printf("%d %d %d\n", a, b, c);` 程序运行结果不会改变。 ()

xxx 选择题

5. 5) 若输入数据为 3 7 3, 则输出为 ()。
 A. 7 3 3 B. 7 3 2 C. 3 7 3 D. 3 7 2
6. 6) 若将第 10 行 $c = a / b$ 改成 $c = b * a$, 输入 0 4 3, 则输出是 ()。
 A. 4 0 0 B. 4 0 3 C. 0 4 0 D. 程序出错无输出

2.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 int n, m, result, a[101], b[101];
4 int power(int x, int y)
5 {
6     int i, ret = 1;
7     for (i = 1; i <= y; ++i)
8         ret *= x;

```

```

9     return ret;
10 }
11 int main()
12 {
13     int i = 2;
14     cin >> n;
15     while (1 != n)
16     {
17         if (0 == n % i)
18         {
19             ++m;
20             a[m] = i;
21             while (0 == n % i)
22             {
23                 ++b[m];
24                 n /= i;
25             }
26         }
27         ++i;
28     }
29     result = 1;
30     for (i = 1; i <= m; ++i)
31     {
32         result *= (a[i] - 1) * power(a[i], b[i] - 1);
33     }
34     cout << result;
35 }

```

xxx 判断题

1. 1) 不管输入什么值,程序的输出值一定比输入值大。 ()
2. 2) 若输入 n 是一个质数,则输出值 $result$ 也可能是一个质数。 ()
3. 3) 若输入 n 是一个质数,程序执行到第 29 行时, i 的值等于刚开始输入的 n 。 ()
4. 4) 若输入 n 的值等于正整数 c 的正整数 d 次方 c^d ($c, d \geq 2$), 程序的第 15 行循环执行完成后, m 的值为 1, $a[1]$ 的值为 c 。 ()

xxx 选择题

5. 5) 若输入 $n = 2400$, 则输出 $result$ 为 ()。
A. 640 B. 1200 C. 2400 D. 960
6. 6) 若输出 $result = 2400$, 则输入 n 可能为 ()。

A. 3200

B. 9000

C. 6400

D. 7200

3.

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4 int main()
5 {
6     string str;
7     char ch1, ch2;
8     getline(cin, str);
9     ch1 = ch2 = ' ';
10    for (int i = 0; i < str.size(); ++i)
11        if (ch1 < str[i])
12            {
13                ch2 = ch1;
14                ch1 = str[i];
15            }
16        else if (ch2 < str[i])
17            ch2 = str[i];
18    if (isalpha(ch1))
19        ch1 ^= ' ';
20    if (isdigit(ch2))
21        ch2 = ch2 - '0' + 48;
22    cout << ch1 << " " << int(ch2) << endl;
23    return 0;
24 }
```

xxx 判断题

1. 1) 若将第 8 行替换为 `cin >> str;` ,程序运行结果不会改变。 ()
2. 2) 若输入字符串只包含大写字母,则输出 `int(ch2)` 一定小于 91。 ()
3. 3) 若将第 19 行替换为 `ch1 ^= 32;` ,程序运行结果不会改变。 ()
4. 4) 若输入 555555,程序运行结果输出为 5 5。 ()

xxx 选择题

5. 5) 若输入数据为 654321,程序运行结果输出为()。

A. 6 5

B. 6 53

C. 1 2

D. 6 1

6.6) 若输入只有 3 个数字型字符,输出为 3 50,则输入一共有多少种不同方案()。

- A. 16 B. 24 C. 8 D. 32

7.7) 若输入数据为 *The 19th Asian Games Hangzhou 2022*,则输出是()。

- A. Z 117 B. z 117 C. Z 9 D. z 9

三、完善程序(单选题, 每小题 3 分, 共计 30 分)

1.

(最少问题) 我们玩一个按字符的游戏,有一组只有 0 和 1 二进制数组成的字符串,每按下其中一个字符,其相邻的字符和它本身都会变成和原来不同的字符(0 变 1, 1 变 0)。如果按下的字符非最左边和最右边字符,则其本身和左右相邻的两个字符变成和原来不同的字符;如果按下最左边字符,则其本身和右边相邻的一个字符变成和原来不同的字符;如果按下最右的字符,则其本身和左边相邻的一个字符变成和原来不同的字符。

给出一张“初始字符串”和一张“最终字符串”。通过按下字符,使“初始字符串”变为“最终字符串”,求最少需要按几次可以完成。如:初始字符串为 010,最终字符串为 110。首先按下 2 号字符,3 个字符变为 101,然后按下 3 号字符,3 个字符变为 110,故使“初始字符串”变为“最终字符串”最少需要按 2 次。

如下所述步骤: 010 $\rightarrow$ 101 $\rightarrow$ 110

输入描述:

第一行输入一个由“0”和“1”组成的字符串,字符串长度为 $n(1 < n < 100)$,表示游戏初始字符串状态。第二行输入一个由“0”和“1”组成的字符串,字符串长度为 $n(1 < n < 100)$,表示游戏最终字符串状态。

输出描述:

输出一个整数,如果通过按字符不能使初始字符串变为最终字符串,则输出 0, 否则输出最少需要按几次字符可以使初始字符串变为最终字符串。

样例输入 1

010

110

样例输出 1

2

样例输入 2

00

10

样例输出 2

0

```
1 #include <bits/stdc++.h>
```

```
2 using namespace std;
```

```
3 string a, b, temp;
```

```

4 int MIN = 0, push_cnt;
5 void change(int i) // 改变当前字符的值
6 {
7     if (i >= 0 && ①)
8     {
9         if (②)
10             temp[i] = '1';
11         else
12             temp[i] = '0';
13     }
14 }
15 void push(int i) // 按下当前位置的字符产生的影响
16 {
17     change(i - 1);
18     change(i);
19     change(i + 1);
20     ③;
21 }
22 void traverse()
23 {
24     for (int i = 1; i < a.size(); i++)
25     {
26         if (④)
27         {
28             push(i);
29         }
30     }
31     if (temp == b && push_cnt < MIN)
32         ⑤;
33 }
34 int main()
35 {
36     cin >> a >> b;
37     MIN = 0x3F3F3F3F;
38     // 第1种情况,按第1个字符
39     push_cnt = 0;
40     temp = a;
41     push(0);
42     traverse();
43     // 第2种情况,不按第1个字符
44     push_cnt = 0;
45     temp = a;

```

```

46     traverse();
47     if (MIN == 0x3F3F3F3F)
48         cout << 0;
49     else
50         cout << MIN;
51     return 0;
52 }

```

1. 1) ①处应填 ()

A. $i < a.size()$ B. $i \leq a.size()$ C. $i \leq a.length()$ D. $i \leq sizeof(a)$

2. 2) ②处应填 ()

A. $'0' == temp[i-1]$ B. $'0' == temp[i]$
 C. $'0' == temp[i+1]$ D. $0 == temp[i]$

3. 3) ③处应填 ()

A. $push_cnt++$ B. $push_cnt--$
 C. $MIN = push_cnt + 1$ D. $MIN = push_cnt$

4. 4) ④处应填 ()

A. $temp[i-1] != a[i-1]$ B. $temp[i-1] == a[i-1]$
 C. $temp[i-1] != b[i-1]$ D. $temp[i-1] == b[i-1]$

5. 5) ⑤处应填 ()

A. $push_cnt++$ B. $push_cnt--$
 C. $MIN = push_cnt + 1$ D. $MIN = push_cnt$

2.

在大数据时代数据爆炸性的增长,在海量数据处理中,我们经常会遇到的某几类问题,一种是优先级队列,一种在给定演艺数据中找出出现频率最高的前 n 个演员,或者从电影数据中找出最近一个月热度最高的前 m 部电影,这类问题通常被称为 *top K* 问题。例如,在搜索引擎中,统计搜索最热门的前 10 大足球明星或者 *NBA* 篮球明星,在金曲榜中统计下载最高的前 10 首流行歌曲等。下面的代码是一个经典的堆排序过程,请补充完整全部程序。

```

1 #include <iostream>
2 #include <algorithm>
3 using namespace std;
4 void heap_change(int *arr, int i, int len)
5 {

```



```

6    int l_kid = 2 * i;
7    int r_kid = 2 * i + 1;
8    int max = i;
9    if (i <= len / 2)
10   {
11       if ( ① )
12       {
13           max = l_kid;
14       }
15       if ( ② )
16       {
17           max = r_kid;
18       }
19       if ( ③ )
20       {
21           swap(arr[i], arr[max]);
22           ④;
23       }
24   }
25 }
26 void build_heap(int *arr, int len)
27 {
28     for (int i = len / 2; i >= 1; --i)
29     {
30         heap_change(arr, i, len);
31     }
32 }
33 void heap_sort(int *arr, int len)
34 {
35     int i;
36     build_heap(arr, len);
37     for (i = len; i >= 1; --i)
38     {
39         swap(arr[1], arr[i]);
40         ⑤;
41     }
42 }
43 void print_value(int *a, int len)
44 {
45     for (int i = 1; i <= len; ++i)
46     {
47         cout << a[i] << " ";

```

```

48     }
49     cout << endl;
50 }
51 int main()
52 {
53     int n;
54     cin >> n;
55     int a[n + 1];
56     for (int i = 1; i <= n; i++)
57     {
58         cin >> a[i];
59     }
60     print_value(a, n);
61     heap_sort(a, n);
62     print_value(a, n);
63     return 0;
64 }

```

1. 1) ①处应填 ()

- A. $l_kid > len \ \&\& \ arr[l_kid] > arr[max]$
- B. $l_kid \leq len \ \&\& \ arr[l_kid] > arr[max]$
- C. $r_kid > len \ \&\& \ arr[r_kid] > arr[max]$
- D. $r_kid \leq len \ \&\& \ arr[r_kid] > arr[max]$

2. 2) ②处应填 ()

- A. $l_kid > len \ \&\& \ arr[l_kid] > arr[max]$
- B. $l_kid \leq len \ \&\& \ arr[l_kid] > arr[max]$
- C. $r_kid > len \ \&\& \ arr[r_kid] > arr[max]$
- D. $r_kid \leq len \ \&\& \ arr[r_kid] > arr[max]$

3. 3) ③处应填 ()

- A. max
- B. $max \neq i$
- C. $max \% 2$
- D. $max \leq size$

4. 4) ④处应填 ()

- A. $heap_change(arr, i, len)$
- B. $heap_change(arr, max, len)$
- C. $heap_change(arr, max - 1, len)$
- D. $heap_change(arr, max, len/2)$

5. 5) ⑤处应填 ()

- A. $heap_change(arr, 1, i)$
- B. $heap_change(arr, 1, i + 1)$

C. *heap_change*(*arr*, 1, *i* − 1)

D. *heap_change*(*arr*, 1, *len*)